

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Database Management Systems
COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO3: *Analyze relational databases using functional dependencies, normalization (1NF to 5NF), and key constraints to identify redundancy and ensure data integrity.*
(BL2, BL3)

Activity Tool : Experiments (High)
Assessment Tool Weightage: 40%

Dr.G.Ayyappan

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	For the relation STUDENT_COURSE(SID, SName, CID, CName, Instructor, Grade), identify all functional dependencies and determine the candidate keys.	BL2 – Understand	5
2	Demonstrate the process of converting an unnormalized relation into 1NF with step-by-step justification and example.	BL3 – Apply	5
3	Given relation R(A,B,C,D,E) with FDs: $A \rightarrow BC$, $BC \rightarrow D$, $D \rightarrow E$. Analyze and decompose it to 2NF eliminating partial dependencies.	BL3 – Apply	5
4	Apply 3NF decomposition on the relation EMPLOYEE(EmpID, EmpName, DeptID, DeptName, ManagerID) with transitive dependencies.	BL3 – Apply	5
5	Verify whether R(A,B,C,D) with FDs: $AB \rightarrow C$, $C \rightarrow D$, $D \rightarrow A$ is in BCNF. If not, decompose it into BCNF.	BL3 – Apply	5
6	Experimentally verify lossless-join decomposition for a given relation using the tabular (Chase) algorithm.	BL3 – Apply	5

7	Identify the multi-valued dependencies in TEACHER(TID, Subject, Hobby) and decompose it into 4NF.	BL3 – Apply	5
8	Demonstrate how join dependencies lead to redundancy. Decompose a given relation to 5NF (Project-Join Normal Form).	BL3 – Apply	5
9	Given a poorly designed database schema for a college, identify all anomalies (insertion, deletion, update) and normalize it to 3NF.	BL2 – Understand	5
10	Compute the closure of attribute sets for the given FDs and determine all candidate keys for the relation.	BL3 – Apply	5

Code of Conduct:

I V. Sujithra (192521445) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

Question 1

Identify all Functional Dependencies and determine the Candidate Keys for the relation STUDENT_COURSE (SID, SName, CID, CName, Instructor, Grade).

Definition

A Functional Dependency (FD) is a relationship between two attributes in a relation where the value of one attribute uniquely determines the value of another attribute. Functional dependencies are used to identify redundancy and to normalize database tables.

A Candidate Key is the minimum set of one or more attributes that can uniquely identify each tuple (record) in a relation. A relation may have one or more candidate keys, but each candidate key must uniquely identify every row.

Given Relation

STUDENT_COURSE (SID, SName, CID, CName, Instructor, Grade)

Where:

- **SID** – Student ID
- **SName** – Student Name
- **CID** – Course ID
- **CName** – Course Name
- **Instructor** – Faculty handling the course
- **Grade** – Grade obtained by the student

Functional Dependencies

From the given relation, the functional dependencies are:

1. **SID $\rightarrow$ SName**
 - Every Student ID uniquely identifies the Student Name.
2. **CID $\rightarrow$ CName, Instructor**
 - Every Course ID uniquely identifies the Course Name and the Instructor handling the course.
3. **SID, CID $\rightarrow$ Grade**
 - The Grade depends on both the Student ID and the Course ID because one student can study many courses and obtain different grades in each course.

Explanation

Each student has a unique Student ID; therefore, SID determines the student's name.

Similarly, every course has a unique Course ID. Hence, CID determines the course name and instructor.

The grade cannot be determined using only SID because a student may enroll in multiple courses. Likewise, CID alone cannot determine the grade because many students can take the same course. Therefore, the combination of **SID and CID** uniquely determines the Grade.

Candidate Key Calculation

Consider the attribute set:

{SID, CID}⁺

Using the functional dependencies:

- $SID \rightarrow SName$
- $CID \rightarrow CName, Instructor$
- $SID, CID \rightarrow Grade$

Therefore,

$\{SID, CID\}^+ = \{SID, SName, CID, CName, Instructor, Grade\}$

Since all attributes of the relation are obtained, **(SID, CID)** is the **Candidate Key**.

Example Table

SID SName CID CName Instructor Grade

101 Rahul C101 DBMS Dr. Kumar A

101 Rahul C102 Java Dr. Ravi B+

102 Priya C101 DBMS Dr. Kumar A+

From the above table:

- **$101 \rightarrow Rahul$**
- **$C101 \rightarrow DBMS, Dr. Kumar$**
- **$(101, C101) \rightarrow A$**

Advantages of Identifying Functional Dependencies

- Helps in database normalization.
- Eliminates data redundancy.
- Prevents update, insertion, and deletion anomalies.
- Improves data consistency.
- Maintains data integrity.

Conclusion

The functional dependencies in the relation are:

- **SID $\rightarrow$ SName**
- **CID $\rightarrow$ CName, Instructor**
- **SID, CID $\rightarrow$ Grade**

The **Candidate Key is (SID, CID)** because it uniquely identifies every record in the STUDENT_COURSE relation. Identifying these dependencies helps in designing an efficient and normalized database.

Question 2

Demonstrate the process of converting an unnormalized relation into First Normal Form (1NF) with step-by-step justification and example.

Definition

Normalization is the process of organizing data in a database to reduce redundancy and improve data integrity. It divides large tables into smaller related tables and establishes relationships among them.

First Normal Form (1NF) is the first stage of normalization. A relation is said to be in 1NF if:

- Every attribute contains only atomic (single) values.
- There are no repeating groups or multi-valued attributes.
- Each row is unique.

Unnormalized Relation (UNF)

Consider the following STUDENT table:

StudentID	StudentName	Subjects
101	Rahul	DBMS, Java, Python
102	Priya	DBMS, C
103	Kiran	Java, Python

In this table, the **Subjects** column contains multiple values in a single cell. Therefore, the relation is **not in First Normal Form**.

Problems in UNF

- One attribute stores multiple values.
- Difficult to search individual subjects.
- Data redundancy increases.
- Updating subject details becomes difficult.
- Violates the rule of atomicity.

Step 1: Identify the Repeating Group

The repeating group is:

Subjects = {DBMS, Java, Python}

Since one attribute contains multiple values, it must be separated.

Step 2: Convert into 1NF

Split each subject into a separate row.

StudentID	StudentName	Subject
101	Rahul	DBMS
101	Rahul	Java
101	Rahul	Python
102	Priya	DBMS
102	Priya	C

103	Kiran	Java
103	Kiran	Python

Now every column contains only **one value**.

Explanation

In the original table, the **Subjects** column stored multiple values, making it difficult to retrieve, update, or delete information.

After converting the table into 1NF:

- Each subject is stored in a separate row.
- Every attribute contains only one value.
- The relation follows the rule of atomicity.
- Data becomes easier to search and manage.

Advantages of First Normal Form (1NF)

- Eliminates repeating groups.
- Stores only atomic values.
- Improves data consistency.
- Makes searching and updating easier.
- Forms the foundation for higher normal forms (2NF, 3NF, BCNF).

Example

Before 1NF:

StudentID	StudentName	Subjects
101	Rahul	DBMS, Java

After 1NF:

StudentID	StudentName	Subject
101	Rahul	DBMS
101	Rahul	Java

This clearly shows how a multi-valued attribute is converted into atomic values.

Conclusion

The given unnormalized relation has been successfully converted into **First Normal Form (1NF)** by removing repeating groups and ensuring that every attribute contains only a single value. Therefore, the relation now satisfies the rules of **1NF** and provides better data consistency and easier database management.

Question 3

Given relation R(A, B, C, D, E) with Functional Dependencies: $A \rightarrow BC$, $BC \rightarrow D$, $D \rightarrow E$. Analyze and decompose it to 2NF eliminating partial dependencies.

Definition

Second Normal Form (2NF) is the second stage of normalization. A relation is said to be in 2NF if:

- It is already in First Normal Form (1NF).
- Every non-prime attribute is fully functionally dependent on the whole candidate key.
- There should be no partial dependency.

A partial dependency occurs when a non-key attribute depends only on a part of a composite candidate key.

Given Relation

R(A, B, C, D, E)

Functional Dependencies

- $A \rightarrow BC$
- $BC \rightarrow D$
- $D \rightarrow E$

Step 1: Find the Candidate Key

Calculate the closure of A.

$$A^+ = \{A\}$$

Using $A \rightarrow BC$

$$A^+ = \{A, B, C\}$$

Using $BC \rightarrow D$

$A^+ = \{A, B, C, D\}$

Using $D \rightarrow E$

$A^+ = \{A, B, C, D, E\}$

Since A determines all attributes,

Candidate Key = A

Step 2: Check for Partial Dependency

A partial dependency exists only when the candidate key consists of more than one attribute.

Here,

Candidate Key = A

Since it contains only one attribute, **partial dependency cannot exist.**

Therefore,

The relation already satisfies **Second Normal Form (2NF)**.

Dependency Diagram



Decomposition

Since there are **no partial dependencies**, decomposition is **not required**.

The relation remains

R(A, B, C, D, E)

Explanation

Attribute **A** uniquely determines **B** and **C**.

The combination **BC** determines **D**.

Attribute **D** further determines **E**.

Although there is a chain of dependencies, there is **no partial dependency** because the candidate key contains only one attribute.

Hence, the relation already satisfies the conditions of **Second Normal Form (2NF)**.

Advantages of 2NF

- Eliminates partial dependency.
- Reduces redundancy.
- Improves data consistency.
- Prevents update anomalies.
- Makes the database easier to maintain.

Example

Suppose

StudentID CourseID StudentName

If the candidate key is **(StudentID, CourseID)** and

$\text{StudentID} \rightarrow \text{StudentName}$

then **StudentName** depends only on part of the key, causing a **partial dependency**.

Such a relation is **not in 2NF**.

In the given relation, the candidate key is only **A**, so this problem does not occur.

Conclusion

The candidate key of the relation is **A**. Since the candidate key consists of a single attribute, **no partial dependency exists**. Therefore, the relation is already in **Second Normal Form (2NF)**, and no decomposition is required.

Question 4

Apply 3NF decomposition on the relation EMPLOYEE(EmpID, EmpName, DeptID, DeptName, ManagerID) with transitive dependencies.

Definition

Third Normal Form (3NF) is the third stage of database normalization. A relation is in 3NF if:

- It is already in Second Normal Form (2NF).
- There are no transitive dependencies.
- Every non-key attribute depends only on the candidate key.

A transitive dependency occurs when a non-key attribute depends on another non-key attribute instead of directly depending on the primary key.

Given Relation

EMPLOYEE(EmpID, EmpName, DeptID, DeptName, ManagerID)

Functional Dependencies

- $\text{EmpID} \rightarrow \text{EmpName}, \text{DeptID}$
- $\text{DeptID} \rightarrow \text{DeptName}, \text{ManagerID}$

Step 1: Identify the Candidate Key

Since every employee has a unique Employee ID,

Candidate Key = EmpID

Step 2: Identify the Transitive Dependency

From the functional dependencies,

$\text{EmpID} \rightarrow \text{DeptID}$

$\text{DeptID} \rightarrow \text{DeptName}, \text{ManagerID}$

Therefore,

$\text{EmpID} \rightarrow \text{DeptName}, \text{ManagerID}$ (Indirectly)

This is called a **Transitive Dependency**.

Step 3: Decompose into 3NF

Relation 1

EMPLOYEE(EmpID, EmpName, DeptID)

Primary Key: **EmpID**

Relation 2

DEPARTMENT(DeptID, DeptName, ManagerID)

Primary Key: **DeptID**

Explanation

The department information is repeated for every employee working in the same department. This creates redundancy.

By creating a separate **DEPARTMENT** table, department details are stored only once. The **EMPLOYEE** table stores only the department ID, reducing redundancy and improving consistency.

Example

Before 3NF

EmpID	EmpName	DeptID	DeptName	ManagerID
101	Rahul	D01	CSE	M101
102	Priya	D01	CSE	M101
103	Arun	D02	IT	M102

The values **DeptName** and **ManagerID** are repeated.

After 3NF

EMPLOYEE

EmpID	EmpName	DeptID
101	Rahul	D01
102	Priya	D01
103	Arun	D02

DEPARTMENT

DeptID	DeptName	ManagerID
D01	CSE	M101
D02	IT	M102

Advantages of 3NF

- Removes transitive dependency.
- Reduces redundancy.
- Improves consistency.
- Prevents update, insertion and deletion anomalies.
- Makes database maintenance easier.

Conclusion

The relation is successfully decomposed into **EMPLOYEE** and **DEPARTMENT** tables. This removes transitive dependencies and satisfies the conditions of **Third Normal Form (3NF)**.

Question 5

Verify whether **R(A, B, C, D)** with FDs: **$AB \rightarrow C$, $C \rightarrow D$, $D \rightarrow A$** is in BCNF. If not, decompose it into BCNF.

Definition

Boyce-Codd Normal Form (BCNF) is a stronger version of 3NF.

A relation is in BCNF if every determinant is a super key.

Given Relation

R(A, B, C, D)

Functional Dependencies

- **$AB \rightarrow C$**

- $C \rightarrow D$
- $D \rightarrow A$

Step 1: Find Candidate Key

Compute the closure of **AB**.

AB^+

$= AB$

$AB \rightarrow C$

$= ABC$

$C \rightarrow D$

$= ABCD$

Therefore,

Candidate Key = AB

Step 2: Check BCNF Condition

FD 1

$AB \rightarrow C$

AB is a Candidate Key.

Satisfies BCNF.

FD 2

$C \rightarrow D$

C is **not** a Candidate Key.

Violates BCNF.

FD 3

$D \rightarrow A$

D is **not** a Candidate Key.

Violates BCNF.

Hence the relation is **not in BCNF**.

BCNF Decomposition

Relation 1

R1(C, D)

Relation 2

R2(A, B, C)

Now every determinant is a candidate key in its respective relation.

Hence both relations satisfy BCNF.

Advantages of BCNF

- Eliminates redundancy.
- Prevents update anomalies.
- Ensures every determinant is a super key.
- Improves database consistency.

Conclusion

Since **C** and **D** are not super keys, the relation violates BCNF. After decomposition into **R1(C,D)** and **R2(A,B,C)**, the relation satisfies **Boyce-Codd Normal Form (BCNF)**.

Question 6

Experimentally verify lossless-join decomposition for a given relation using the Tabular (Chase) Algorithm.

Definition

A Lossless Join Decomposition ensures that when a relation is decomposed into two or more smaller relations, the original relation can be reconstructed using a natural join without losing any information.

The Chase Algorithm (Tabular Method) is used to verify whether the decomposition is lossless.

Example

Consider the relation:

R(A, B, C)

Functional Dependency:

$A \rightarrow B$

Suppose the relation is decomposed into:

- R1(A, B)
- R2(A, C)

Tabular Representation

Relation	A	B	C
R1	a1	b1	c1
R2	a1	b2	c2

Since the common attribute is **A**, apply the functional dependency $A \rightarrow B$.

After applying the dependency, **B** becomes the same in both rows.

Relation	A	B	C
R1	a1	b1	c1
R2	a1	b1	c2

Since one row becomes consistent after applying the dependency, the decomposition is **Lossless Join**.

Explanation

The common attribute **A** acts as the connecting attribute between the two decomposed relations. Because of the functional dependency $A \rightarrow B$, the original relation can be reconstructed exactly using a natural join.

Advantages

- No data loss.
- Original relation can be reconstructed.

- Improves database efficiency.
- Reduces redundancy.
- Supports normalization.

Conclusion

Using the Chase Algorithm, the decomposition is verified to be **Lossless Join** because the original relation can be reconstructed without losing any information.

Question 7

Identify the Multi-Valued Dependencies in TEACHER(TID, Subject, Hobby) and decompose it into Fourth Normal Form (4NF).

Definition

Fourth Normal Form (4NF) is the fourth stage of database normalization. A relation is in 4NF if it is already in BCNF and contains no non-trivial Multi-Valued Dependencies (MVDs).

A Multi-Valued Dependency (MVD) exists when one attribute determines multiple independent values of another attribute.

Given Relation

TEACHER(TID, Subject, Hobby)

Where:

- **TID** – Teacher ID
- **Subject** – Subject taught by the teacher
- **Hobby** – Hobby of the teacher

Multi-Valued Dependencies

- **TID $\twoheadrightarrow$ Subject**
- **TID $\twoheadrightarrow$ Hobby**

A teacher can teach multiple subjects and also have multiple hobbies. These two sets of values are independent of each other.

Example

TID	Subject	Hobby
T101	DBMS	Reading
T101	DBMS	Music
T101	Java	Reading
T101	Java	Music

The table contains unnecessary repetitions because subjects and hobbies are stored together.

Decomposition into 4NF

TEACHER_SUBJECT(TID, Subject)

<i>TID</i>	<i>Subject</i>
T101	DBMS
T101	Java

TEACHER_HOBBY(TID, Hobby)

TID	Hobby
T101	Reading
T101	Music

Advantages

- Eliminates multi-valued dependencies.
- Reduces redundancy.
- Prevents insertion, update, and deletion anomalies.
- Improves data consistency.
- Makes database easier to maintain.

Conclusion

The relation is successfully decomposed into **TEACHER_SUBJECT** and **TEACHER_HOBBY**, removing all multi-valued dependencies. Hence, the relation satisfies **Fourth Normal Form (4NF)**.

Question 8

Demonstrate how Join Dependencies lead to redundancy. Decompose a given relation to Fifth Normal Form (5NF).

Definition

Fifth Normal Form (5NF), also known as Project-Join Normal Form (PJNF), is the highest level of normalization. A relation is in 5NF if every join dependency is implied by the candidate keys and no further decomposition can be performed without losing information.

Example Relation

PROJECT(Employee, Project, Skill)

Suppose:

- One employee works on many projects.
- One employee has many skills.
- A project requires many skills.

This creates **Join Dependency** and redundancy.

Sample Table

Employee	Project	Skill
Rahul	Library	Java
Rahul	Library	SQL
Rahul	Banking	Java

The same information is repeated several times.

5NF Decomposition

EMPLOYEE_PROJECT(Employee, Project)

Employee	Project
Rahul	Library
Rahul	Banking

EMPLOYEE_SKILL(Employee, Skill)

Employee	Skill
Rahul	Java
Rahul	SQL

PROJECT_SKILL(Project, Skill)

Project	Skill
Library	Java
Library	SQL
Banking	Java

Advantages

- Removes join redundancy.
- Eliminates duplicate data.
- Improves consistency.
- Supports efficient joins.
- Prevents anomalies.

Conclusion

By decomposing the relation into three smaller relations, join dependency is removed and the relation satisfies **Fifth Normal Form (5NF)**.

Question 9

Given a poorly designed database schema for a college, identify all anomalies (Insertion, Deletion, Update) and normalize it to Third Normal Form (3NF).

Definition

A poorly designed database often contains redundant data, which leads to anomalies. Normalization removes redundancy and improves data integrity.

Example Relation

StudentID	StudentName	Department	HOD
101	Rahul	CSE	Dr. Kumar
102	Priya	CSE	Dr. Kumar
103	Arun	IT	Dr. Ravi

Insertion Anomaly

A new department cannot be added unless at least one student is admitted to that department.

Update Anomaly

If the HOD of the CSE department changes, every row containing CSE must be updated.

Deletion Anomaly

If the last student in the IT department is deleted, the information about the IT department and its HOD is also lost.

Normalization to 3NF

STUDENT(StudentID, StudentName, DeptID)

StudentID	StudentName	DeptID
101	Rahul	D01
102	Priya	D01
103	Arun	D02

DEPARTMENT(DeptID, DepartmentName, HOD)

DeptID	DepartmentName	HOD
D01	CSE	Dr. Kumar
D02	IT	Dr. Ravi

Advantages

- Eliminates redundancy.
- Removes anomalies.
- Improves consistency.
- Simplifies maintenance.
- Maintains data integrity.

Conclusion

The relation is successfully normalized into **Third Normal Form (3NF)** by separating student and department information, thereby eliminating insertion, update, and deletion anomalies.

Question 10

Compute the Closure of Attribute Sets for the given Functional Dependencies and determine all Candidate Keys for the relation.

Definition

The Closure of an Attribute Set is the set of all attributes that can be functionally determined from a given attribute or attribute set using the functional dependencies.

A Candidate Key is the smallest set of attributes that uniquely identifies every tuple in a relation.

Given Functional Dependencies

Assume:

- $A \rightarrow B$
- $B \rightarrow C$
- $C \rightarrow D$

Relation:

R(A, B, C, D)

Step 1: Compute Attribute Closure

Calculate A^+

Initially,

$$A^+ = \{A\}$$

Using $A \rightarrow B$

$$A^+ = \{A, B\}$$

Using $B \rightarrow C$

$$A^+ = \{A, B, C\}$$

Using $C \rightarrow D$

$$A^+ = \{A, B, C, D\}$$

Since all attributes are obtained,

$$A^+ = \{A, B, C, D\}$$

Step 2: Determine Candidate Key

Since the closure of **A** contains every attribute in the relation,

Candidate Key = A

Example

Attribute	Closure
A	{A,B,C,D}
B	{B,C,D}
C	{C,D}
D	{D}

From the above table, only **A** determines all attributes.

Advantages of Attribute Closure

- Helps identify candidate keys.
- Verifies functional dependencies.
- Supports normalization.
- Reduces redundancy.
- Improves database design.

Conclusion

The closure of attribute **A** is **{A, B, C, D}**, which includes all attributes of the relation. Therefore, **A** is the **Candidate Key**. Attribute closure is an important concept used in normalization and database design.